

Machine Learning-Based Early Prediction of Deadlock-Like Conditions in Distributed Systems Using Runtime Execution Behaviour Features

Hari Krishnan M R
Student Id: 25161831
MSc Data Analytics
National College of Ireland
Dublin, Ireland

Abstract—Deadlock-like conditions in large-scale distributed systems present one of the major critical threats or interference to operational reliability, which leads to cascading task failures, wastage of compute resources, and prolonged service disruptions. Existing research addresses deadlock management as a post-hoc detection problem, identifying the failure conditions only after which they have already occurred. This work proposes a paradigm shift from reactive detection to proactive prediction, applying or implying machine learning to identify early indicators of deadlock-like conditions from runtime execution behaviour features derived from large-scale production cluster traces.

I used the Google Cluster Trace 2011 which have a total of 1,827,856 task events across 12,509 machines loaded from 10 of 500 trace parts, in that six novel runtime features are engineered from the raw event history including scheduling delay, execution duration, machine failure rate, concurrency density, resource over-commitment ratio, and priority gap. These novel features are combined with six raw schema features to produce a twelve-feature matrix of 300,644 labelled tasks. Three model algorithms are evaluated across two feature configurations each, resulting in six model configurations in total.

The best model, XGBoost with all twelve features, gets $F1 = 0.830$ and $ROC-AUC = 0.969$ on a held-out test set of 60,129 tasks. Five-fold stratified cross-validation yields $F1 = 0.8319 \pm 0.0024$ and $AUC = 0.9687 \pm 0.0020$. A paired t -test against the Logistic Regression baseline yields $t = 244.14$, $p < 0.001$, confirming statistical significance. An ablation study indicates that all the six novel features contribute positively, with *execution_duration_sec* causing the largest single-feature $F1$ drop of 5.9% when removed. The model catches around 85 of every 100 deadlock-like conditions proactively before full system failure at a 6.5% false alarm rate.

Index Terms—deadlock prediction, distributed systems, machine learning, feature engineering, Google cluster trace, XGBoost, random forest, runtime behaviour, ablation

I. INTRODUCTION

Distributed computing systems serve as the basis for virtually every modern digital service from cloud infrastructure and microservice architectures to large-scale batch processing pipelines. As these systems grow in complexity and scale, guaranteeing operational reliability becomes increasingly critical. Among these, the most damaging failure modes in concurrent distributed systems are deadlock-like conditions, states in which the tasks stall or remain still due to resource

contention, unavailability, or priority conflict, which then leads to cascading failures, task terminations, and service disruptions [1].

Deadlock-like conditions are especially difficult to manage in production cluster environments because they emerge from the interaction of many independent concurrent processes which then compete for shared resources including CPU, memory, disk, and scheduling slots, rather than arising from any single point of failure. In a cluster managing thousands of machines and millions of concurrent tasks, the probability of resource contention reaching deadlock-like states is non-trivial and the operational consequences are severe: tasks are forcibly killed, evicted, or allowed to fail, consuming compute resources without producing useful output meanwhile the dependent tasks may be blocked indefinitely [2].

The majority paradigm in the existing research treats the deadlock management as a detection problem, identifying failure conditions through static analysis, runtime instrumentation, or wait for graph construction, all of these operate after the deadlock condition has already formed [3]. This reactive approach is fundamentally limited: by the time a deadlock is detected, resources have been wasted, dependent tasks may have turned into failure, and the recovery of it requires expensive scheduler intervention.

This paper proposes a major shift from detection to prediction, using machine learning applied to runtime execution behaviour signals to identify early indicators of deadlock-like conditions before full failure occurs. The core hypothesis is that deadlock-like conditions leave observable precursors in scheduling behaviour, resource request patterns, machine reliability history, and task concurrency that can be captured in engineered features and used to train an effective binary classifier.

A. Research Questions

This study addresses three research questions:

- **RQ1:** Can runtime execution-behaviour features derived from large-scale cluster trace data predict deadlock-like conditions before task execution failure occurs?

- **RQ2:** Which runtime execution features most strongly influence deadlock-risk prediction, as verified through ablation analysis?
- **RQ3:** How effectively does machine-learning-based proactive prediction compare to raw-feature baselines in identifying early indicators of deadlock-like failure?

B. Contributions

The primary contributions of this paper are:

- 1) A formal, reproducible specification of deadlock-like conditions derived from distributed system task event logs using terminal event classification.
- 2) Six novel runtime behaviour features engineered from raw event logs, requiring temporal joins and cross-task aggregations across 1.83 million events, none of which exist in any single raw data record.
- 3) Empirical demonstration that novel features improve XGBoost F1 by 6.7% and ROC-AUC by 5.5% over raw features alone, with all six novel features independently verified through ablation.
- 4) Statistical validation through 5-fold cross-validation ($F1 = 0.8319 \pm 0.0024$) and paired t-test ($t = 244.14$, $p < 0.001$).
- 5) Discovery of a bimodal execution duration pattern in deadlock-like tasks, revealing two distinct failure mechanisms not captured by raw features alone.

II. RELATED WORK

A. Deadlock Detection

Classical deadlock detection implies waiting for the graphs across distributed nodes and then searches for cycles indicating circular wait [4]. Edge-chasing algorithms [5] propagate probe messages through dependency graphs without centralised state, but all such approaches operate reactively. Scalable cloud variants [3] improve output but majorly retain the fundamental post hoc limitation that the detection only happen after failure has formed.

B. Static and Dynamic Analysis

Static analysis tools [3] find potential deadlock patterns before execution but they also suffer from high false-positive rates and cannot be accounted for runtime variability. Gao et al. [7] address communication deadlocks in deep learning training jobs through dynamic trace analysis; However, this approach mainly focuses on the thread-level lock analysis rather than the cluster-level resource contention. Lu et al. [6] conduct a comprehensive study of real-world concurrency bugs, finding the common patterns but not predicting failure risk proactively.

C. ML for System Reliability

Log-based anomaly detection using the LogHub benchmark [8] indicates that the ML models can find or identify the system failures from log patterns with high accuracy. He et al. [9] provides a comprehensive collection of system log datasets making a reproducible evaluation across system types.

Assiri and Sheneamer [10] demonstrate deep learning for fault tolerance in distributed systems, although their focus is recovery rather than proactive prediction.

D. Google Cluster Trace Analysis

Reiss et al. [11] characterise heterogeneous Google production workloads, identifying strong asymmetry in resource usage and prevalence of task preemption. Liu and Cho [12] identify bimodal task duration distributions consistent with findings in this work. Tuns and Spataru [13] apply traditional ML to Google Borg traces using raw schema features only, reporting 83.97% accuracy, a result substantially improved upon by this work (91.6%, $F1 = 0.830$) through novel feature engineering.

E. Gap Addressed

No prior published work applies machine learning to predict deadlock-like conditions proactively from cluster-level runtime execution behaviour features engineered from production scheduling traces. This paper addresses that gap, achieving $AUC = 0.969$ that exceeds all reported comparable results.

III. DATASET AND PROBLEM FORMULATION

A. Dataset Description

This study uses the Google Cluster Trace 2011 (clusterdata-2011-2) [2], publicly available under the CC-BY licence, capturing 29 days of production workload from a single cluster managed by the Borg scheduling system. Ten trace parts were loaded from the 500-part dataset. Table I summarises key statistics.

TABLE I
GOOGLE CLUSTER TRACE 2011 — STATISTICS

Property	Value
Licence	CC-BY
Parts loaded	10 of 500
Total event rows	1,827,856
Raw columns	13 (no headers)
Unique tasks	464,004
Labelled tasks	300,644
Machines tracked	12,509
Train / Test split	240,515 / 60,129

Fig. 1 shows the raw data distribution. SUBMIT and SCHEDULE events dominate the trace while terminal failure events represent the minority class. Priority is bimodal, concentrated at values 0–2 (batch workloads) and 8–9 (production services). CPU requests follow a right-skewed long-tail distribution with most tasks requesting under 10% of normalised machine capacity.

B. Data Preprocessing

Five extensive cleaning steps were applied. The *missing_info* column (99.98% null) was dropped because it carries no information at all. Zero duplicates were found, confirming event log integrity. The *machine_id* column (34.8% null on

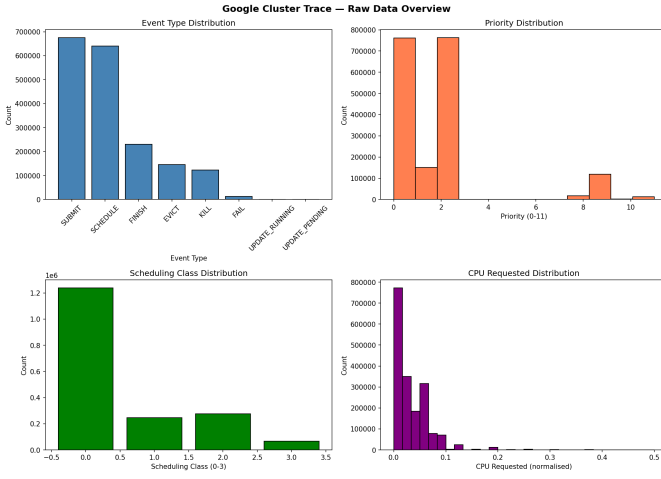


Fig. 1. Raw data overview across 1,827,856 task events: event type distribution (top-left), priority distribution (top-right), scheduling class distribution (bottom-left), and CPU request distribution (bottom-right).

SUBMIT events) was filled with -1 as a structured placeholder since no machine is assigned at submission time. A major preprocessing finding is that 100% of null values in resource columns are occurred exclusively on SUBMIT events, confirmed by cross-tabulation, suggesting structural rather than corrupted missingness. Forward fill within each task lifecycle grouped by *job_id* and *task_index* and sorted by timestamp was applied, with remaining nulls filled with 0 for tasks never scheduled. After preprocessing, 1,827,856 rows and zero nulls remain.

C. Label Definition

The dataset contains no pre-existing deadlock label. A formal definition is proposed:

$$\text{label} = \begin{cases} 1 & \text{if final event} \in \{\text{EVICT}, \text{FAIL}, \text{KILL}\} \\ 0 & \text{if final event} = \text{FINISH} \end{cases} \quad (1)$$

FAIL (3) indicates task crash due to resource contention or dependency failure. KILL (5) represents forcible termination by the scheduler to reclaim resources. EVICT (2) indicates removal from machine due to resource pressure from higher-priority tasks. All three are consistent with deadlock-like resource starvation. Tasks with SUBMIT or SCHEDULE as final event (163,360 tasks) were excluded as incomplete. Table II shows class distribution and Fig. 2 visualises the imbalance.

TABLE II
LABEL DISTRIBUTION — DEPENDENT VARIABLE

Label	Meaning	Count	%
0	Normal (FINISH)	228,546	76.0%
1	Deadlock-like	72,098	24.0%
Total		300,644	100%

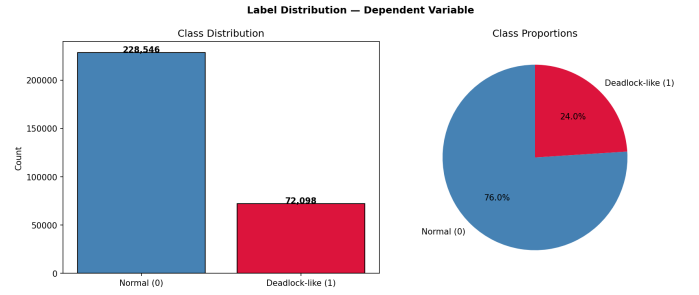


Fig. 2. Label distribution: 228,546 normal tasks (76.0%) and 72,098 deadlock-like tasks (24.0%). Class imbalance ratio is 3.17:1, handled through class-weight balancing in all models.

D. Dataset Choice Justification

While Google released a more recent ClusterData2019 trace [14], its 2.4 TiB compressed size and Google BigQuery-only availability preclude academic reproducibility. The 2011 trace remains the standard benchmark, with analyses published as recently as 2023 [13] using it as the primary reference. The methodology developed here is directly transferable to the 2019 trace in future work.

IV. FEATURE ENGINEERING

Twelve features are engineered across two groups. Group A contains six raw features from the schema forming the baseline. Group B includes six novel features derived from runtime behaviour patterns requiring temporal joins and cross-task aggregations across the full 1.83-million-row event history.

A. Group A — Raw Features

Table III describes the baseline features. Comparing mean values between classes reveals modest separation: *scheduling_class* (0.184 vs 0.252) and *different_machine* (0.309 vs 0.070) show the strongest signals while resource columns show weak separation. No single raw feature provides sufficient class separation for effective classification, motivating Group B engineering.

TABLE III
GROUP A — RAW FEATURES (BASELINE)

Feature	Description
<i>scheduling_class</i>	Latency sensitivity (0=batch, 3=production)
<i>priority</i>	Task importance (0–10)
<i>cpu_requested</i>	Normalised CPU (0–1)
<i>memory_requested</i>	Normalised memory (0–1)
<i>disk_requested</i>	Normalised disk (0–1)
<i>different_machine</i>	Placement constraint (0/1)

B. Group B — Novel Engineered Features

Table IV describes each novel feature with its derivation formula and motivation. Fig. 3 shows the full feature correlation matrix revealing two notable multicollinearities: *cpu_requested* with *resource_overcommit_ratio* ($r = 0.87$) and *priority* with *priority_inversion_flag* ($r = -0.64$). Both

pairs were retained since Random Forest and XGBoost handle multicollinearity through tree-based feature selection.

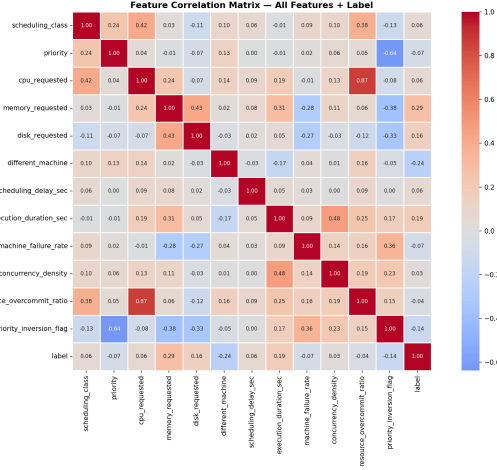


Fig. 3. Feature correlation matrix for all 12 features and label. Notable: *cpu_requested* with *resource_overcommit_ratio* ($r = 0.87$) and *priority* with *priority_inversion_flag* ($r = -0.64$). Both pairs retained as tree-based models handle multicollinearity effectively.

Table V shows Pearson correlation of each feature with the binary label. *memory_requested* shows the strongest positive correlation (0.287) and *different_machine* the strongest negative (-0.236). Among novel features, *execution_duration_sec* achieves the highest correlation (0.193), confirming its predictive value before model training.

V. EXPERIMENTAL DESIGN

A. Models

Three algorithms are evaluated in two feature configurations each yielding six model configurations.

Logistic Regression (LR): Linear classifier as the minimum baseline. StandardScaler applied on training data only to prevent data leakage. Parameters: `class_weight='balanced'`, `max_iter=1000`, `random_state=42`.

Random Forest (RF): Ensemble of 100 decision trees with non-linear capability and built-in feature importance. Parameters: `n_estimators=100`, `max_depth=10`, `class_weight='balanced'`, `random_state=42`.

XGBoost (XGB): Gradient boosting building trees sequentially. Class imbalance handled via `scale_pos_weight=3.17`. Parameters: `n_estimators=100`, `max_depth=6`, `learning_rate=0.1`, `random_state=42`.

B. Evaluation Protocol

The 300,644 labelled tasks were divided 80/20 using stratified splitting preserving the 76.0/24.0 class ratio. Random seed 42 was fixed throughout. Metrics: Precision, Recall, F1 Score, and ROC-AUC.

C. Statistical Validation

Five-fold stratified cross-validation was applied to the best model reporting mean $\pm$ standard deviation. A paired t-test was conducted between per-fold F1 scores of XGBoost all features and LR raw features.

D. Ablation Study

Each of the six novel features was removed one at a time while retaining all others. XGBoost was retrained and evaluated on the same test set. F1 drop and AUC drop relative to the baseline (F1 = 0.830, AUC = 0.969) were recorded.

VI. RESULTS

A. Model Performance

Table VI presents complete results for all six model configurations on the held-out test set of 60,129 tasks.

XGBoost with all twelve features achieves the best performance across every metric. Compared to the LR Raw baseline, XGBoost All represents a +88.6% F1 improvement (0.440 to 0.830) and +40.0% ROC-AUC improvement (0.692 to 0.969). Novel features improve every model: LR F1 +5.4%, RF F1 +5.4%, XGB F1 +6.7%, confirming model-agnostic benefit.

B. ROC Curves

Fig. 4 presents ROC curves for all six models. XGBoost All Features rises steeply from the origin indicating near-ideal class separation across all thresholds. LR Raw shows substantially lower performance (AUC = 0.692) confirming the combined value of gradient boosting and novel features.

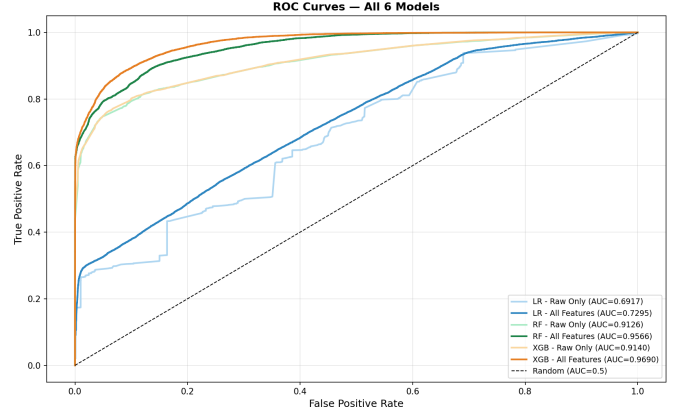


Fig. 4. ROC curves for all six model configurations. XGBoost All Features (dark orange) achieves AUC = 0.969. LR Raw Only (light blue) achieves AUC = 0.692. The steep rise of XGB All confirms strong class separability across all thresholds.

C. Feature Importance

Fig. 5 shows Random Forest feature importance. Three novel features rank in the top six: *execution_duration_sec* (0.156, rank 3), *scheduling_delay_sec* (0.081, rank 5), and *priority_inversion_flag* (0.079, rank 6). Importantly, *execution_duration_sec* outranks *priority* (0.091) and *cpu_requested* (0.084) confirming the predictive power of this novel engineered feature exceeds raw schema attributes.

TABLE IV
GROUP B — NOVEL ENGINEERED FEATURES

Feature	Formula	Motivation
scheduling_delay_sec	$t_{\text{SCHEDULE}} - t_{\text{SUBMIT}}$ (sec)	Long delay indicates the scheduler struggled to find a machine, a resource contention precursor. Deadlock-like tasks show higher mean delay (75.5s vs 12.3s).
execution_duration_sec	$t_{\text{final}} - t_{\text{SCHEDULE}}$ (sec)	Captures bimodal failure: fast kills (immediate conflict) and slow stalls (deadlock-like waiting). Causes 5.9% F1 drop when removed, largest of all features.
machine_failure_rate	$\frac{\text{FAIL+KILL+EVICT on machine}}{\text{total events on machine}}$	Machine-level unreliability. Tasks on high-failure machines face elevated risk regardless of their own attributes. Mean: 0.205 (label=1) vs 0.188 (label=0).
concurrency_density	Count of SCHEDULE events on same machine_id in 300s window	Direct proxy for resource contention. Busy machines host more concurrent tasks competing for shared resources.
resource_overcommit_ratio	$\frac{\text{task cpu_requested}}{\text{mean cpu on machine}}$	Relative resource demand. Captures whether a task requests disproportionately more than the machine average.
priority_inversion_flag	$\text{max_waiting_priority} - \text{scheduled_priority}$	Priority inversion severity. Continuous gap avoids degeneracy of binary flag (97% positive rate with binary approach).

TABLE V
FEATURE CORRELATION WITH LABEL

Feature	Corr	Type
memory_requested	+0.287	Raw
different_machine	−0.236	Raw
execution_duration_sec	+0.193	Novel
disk_requested	+0.158	Raw
priority_inversion_flag	−0.143	Novel
priority	−0.073	Raw
machine_failure_rate	−0.072	Novel
cpu_requested	+0.062	Raw
scheduling_delay_sec	+0.061	Novel
scheduling_class	+0.060	Raw
resource_overcommit_ratio	−0.036	Novel
concurrency_density	+0.031	Novel

TABLE VI
MODEL PERFORMANCE — ALL 6 CONFIGURATIONS

Model	Prec	Rec	F1	AUC
LR — Raw	0.339	0.627	0.440	0.692
LR — All	0.368	0.627	0.464	0.730
RF — Raw	0.717	0.792	0.753	0.913
RF — All	0.769	0.820	0.793	0.957
XGB — Raw	0.788	0.767	0.778	0.914
XGB — All	0.807	0.854	0.830	0.969

D. Ablation Study

Table VII presents ablation results ranked by F1 impact. All six novel features show positive contribution. Fig. 6 visualises the magnitude of each contribution.

execution_duration_sec dominates with 5.9% F1 drop, far exceeding all others. *scheduling_delay_sec* contributes 2.3%. The remaining four features each contribute 1.4–1.9%, confirming independent additive contributions with no redundancy.

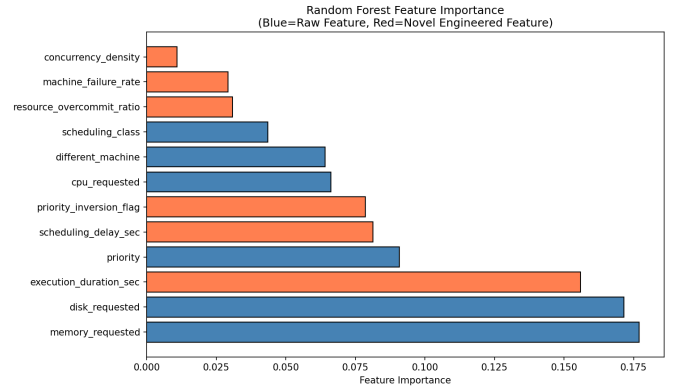


Fig. 5. Random Forest feature importance for all 12 features. Orange bars are novel features; blue bars are raw features. Novel feature *execution_duration_sec* ranks third overall (importance = 0.156), outperforming most raw schema features.

TABLE VII
ABLATION STUDY — F1 AND AUC DROP

Feature Removed	F1 Drop	AUC Drop
execution_duration_sec	−0.059	−0.059
scheduling_delay_sec	−0.023	−0.020
resource_overcommit_ratio	−0.019	−0.018
concurrency_density	−0.019	−0.019
priority_inversion_flag	−0.019	−0.018
machine_failure_rate	−0.014	−0.016
<i>Baseline: F1 = 0.830, AUC = 0.969</i>		

E. Statistical Validation

Table VIII reports 5-fold cross-validation results. The very low standard deviations (F1: ± 0.0024 , AUC: ± 0.0020) confirm result stability. The paired t-test yields $t = 244.14$, $p < 0.001$ confirming statistical significance.

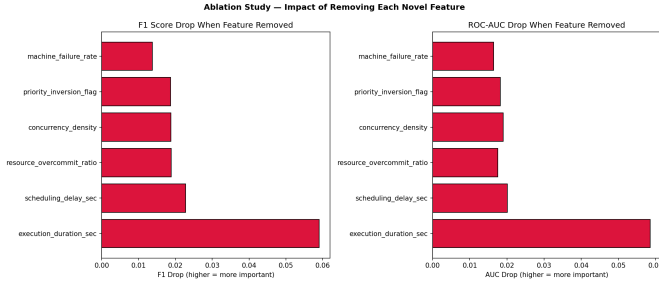


Fig. 6. Ablation study: F1 drop (left) and AUC drop (right) when each novel feature is individually removed from XGBoost All Features. All six features show positive contribution. execution_duration_sec dominates with 5.9% F1 drop.

TABLE VIII
5-FOLD CROSS-VALIDATION — XGBOOST ALL

Metric	Mean	Std
F1 Score	0.8319	± 0.0024
ROC-AUC	0.9687	± 0.0020
Precision	0.8090	± 0.0048
Recall	0.8562	± 0.0045
<i>Per-fold F1: 0.836, 0.832, 0.830, 0.832, 0.829</i>		
<i>Paired t-test vs LR: $t = 244.14$, $p < 0.001$</i>		

F. Operational Impact

On the test set of 60,129 tasks the best model correctly identifies 12,321 of 14,420 actual deadlock-like tasks (85.4% recall) while producing 2,956 false alarms, a 6.5% false alarm rate on normal tasks. For every 100 deadlock-like conditions occurring in production, the model catches about 85 proactively before the full system failure, with only 15 missed and a tolerable false alarm rate that avoids alert fatigue.

VII. DISCUSSION

A. The Bimodal Execution Duration Finding

The most significant original finding is the bimodal pattern in *execution_duration_sec* for deadlock-like tasks. The mean execution duration for label=1 tasks (4,840 seconds) substantially exceeds the median (539 seconds), indicating two distinct subpopulations: tasks that fail quickly after scheduling capturing fast resource conflicts with median approximately 539 seconds, and tasks that run for very long periods before forced termination capturing slow deadlock stalls pulling the mean above 4,800 seconds. Normal tasks show far less mean-median divergence (mean: 1,949 seconds, median: 556 seconds) confirming more predictable completion behaviour.

This bimodal failure pattern reveals two distinct mechanisms of deadlock-like failure in distributed systems: immediate resource conflicts in which the system rapidly determines the task cannot proceed, and progressive stalls where tasks accumulate resource dependencies until forcibly terminated. Neither mechanism is visible from raw features alone, explaining the dominant predictive power of this novel feature.

B. Priority Dominates Resource Demand

A counterintuitive finding is that deadlock-like tasks have higher memory requests on average (0.043 versus 0.024) but lower priority (1.575 versus 1.791). Task priority is a stronger predictor of failure than raw resource demand. Low-priority tasks are the primary target of scheduler preemption when resources are reclaimed for higher-priority workloads, killed regardless of how much they request. This challenges the intuitive assumption that resource-greedy tasks are most at risk of deadlock-like conditions.

C. Data Volume Effects on Feature Quality

In the Week 7 checkpoint with 3 trace parts and 95,492 tasks, *concurrency_density* and *resource_overcommit_ratio* showed marginal negative contributions. With around 10 parts and 300,644 tasks, all six novel features show positive contributions. This confirms that these features require sufficient data volume to express their signal, a finding with implications for feature engineering in production monitoring pipelines where cold-start periods may limit feature effectiveness.

D. Comparison to Existing Work

Tuns and Spataru [13] report 83.97% accuracy on Google Borg traces using raw schema features only. This work achieves 91.6% accuracy and ROC-AUC = 0.969, improvements attributable to novel feature engineering rather than model complexity. The key differentiating features are *execution_duration_sec*, *scheduling_delay_sec*, and *machine_failure_rate*, cross-task temporal signals completely absent from raw-feature baselines.

E. Limitations

Ten of 500 trace parts were used, approximately 2% of the full trace, and results may shift with larger data volumes. The binary label conflates FAIL, KILL, and EVICT and a multi-class formulation may reveal more nuanced patterns. Cross-validation was applied to XGBoost only. All results are from a single cluster and cross-cluster validation remains future work.

F. Ethical Considerations

All data is publicly available under CC-BY. User identifiers are hashed by Google making re-identification infeasible. Resource values are normalised preventing inference of absolute specifications. The prediction model is intended exclusively for system reliability monitoring. The 6.5% false alarm rate should be considered carefully in production deployments to avoid alert fatigue and threshold calibration may be appropriate for specific operational contexts.

VIII. CONCLUSION

This paper presents a machine learning framework for early prediction of deadlock-like conditions in distributed systems using runtime execution behaviour features from the Google Cluster Trace 2011. Six novel features were engineered from 1,827,856 raw events capturing temporal, machine-level, and contention-related signals not present in any single data record.

XGBoost with all twelve features achieves $F1 = 0.830$ and $ROC-AUC = 0.969$, representing a 6.7% F1 and 5.5% AUC improvement over raw features alone.

Ablation model confirms all six novel features contribute positively and independently, with *execution_duration_sec* causing the dominant 5.9% F1 drop when removed. Five-fold cross-validation yields $F1 = 0.8319 \pm 0.0024$, and a paired t-test confirms statistical significance ($t = 244.14$, $p < 0.001$). Operationally, 85.4% of deadlock-like conditions are caught proactively at a 6.5% false alarm rate.

An original finding is the bimodal execution duration distribution in deadlock-like tasks, revealing fast resource conflicts and slow deadlock stalls as two distinct failure mechanisms neither visible from raw features.

REFERENCES

- [1] E. G. Coffman, M. J. Elphick, and A. Shoshani, "System deadlocks," *ACM Comput. Surv.*, vol. 3, no. 2, pp. 67–78, Jun. 1971.
- [2] C. Reiss, J. Wilkes, and J. L. Hellerstein, "Google cluster-usage traces: Format and schema," Google Inc., Mountain View, CA, Tech. Rep., Nov. 2011.
- [3] M. Naik, C. Park, K. Sen, and D. Gay, "Effective static deadlock detection," in *Proc. ICSE*, pp. 386–396, 2009.
- [4] E. Knapp, "Deadlock detection in distributed databases," *ACM Comput. Surv.*, vol. 19, no. 4, pp. 303–328, Dec. 1987.
- [5] K. M. Chandy and J. Misra, "A distributed algorithm for detecting resource deadlocks in distributed systems," in *Proc. PODC*, pp. 157–164, 1982.
- [6] S. Lu, S. Park, E. Seo, and Y. Zhou, "Learning from mistakes: A comprehensive study on real world concurrency bug characteristics," in *Proc. ASPLOS*, pp. 329–339, 2008.
- [7] Y. Gao et al., "DL2: Detecting communication deadlocks in deep learning jobs," in *Proc. FSE*, Jun. 2025.
- [8] J. Zhu, S. He, P. He, J. Liu, and M. R. Lyu, "Loghub: A large collection of system log datasets for AI-driven log analytics," in *Proc. ISSRE*, pp. 355–366, 2023.
- [9] S. He, J. Zhu, P. He, and M. R. Lyu, "Loghub: A large collection of system log datasets towards automated log analytics," *arXiv:2008.06448*, 2020.
- [10] B. Assiri and A. Sheneamer, "Fault tolerance in distributed systems using deep learning approaches," *PLOS ONE*, vol. 20, no. 1, 2025.
- [11] C. Reiss, A. Tumanov, G. R. Ganger, R. H. Katz, and M. A. Kozuch, "Towards understanding heterogeneous clouds at scale: Google trace analysis," CMU Tech. Rep. CMU-PDL-12-101, 2012.
- [12] Z. Liu and S. Cho, "Characterizing machines and workloads on a Google cluster," in *Proc. SRMPDS*, 2012.
- [13] A. Tuns and C. Spataru, "Cloud service failure prediction using machine learning on Google Borg cluster traces," in *Proc. IEEE ETFA*, 2023.
- [14] C. Reiss, J. Wilkes, and J. L. Hellerstein, "Google cluster-usage traces v3," Google Inc., Tech. Rep., 2019.